

Projeto Final: Sistema de Inspeção Visual Automática para Classificação de Itens Fora de Padrão

1 Apresentação

Este projeto vale **5,0 pontos**, correspondentes a **50% da nota** da disciplina, e consolida o conteúdo das Aulas. O foco central é o pipeline clássico de visão computacional:

imagem

- > segmentação ou isolamento do objeto
- > extração de features manuais
- > tabela X
- > vetor y
- > classificador clássico
- > avaliação

Cada equipe escolhe **um cenário** entre as opções da Seção 3 e desenvolve um sistema completo de **inspeção visual automática**, comparando **pelo menos dois classificadores clássicos**. MLP, CNN pré-treinada com transfer learning e XAI são bem-vindos, mas opcionais (ver níveis de complexidade na Seção 9).

- **Equipes:** 3 a 4 alunos.
 - **Plataforma de submissão:** Blackboard.
 - **Repositório GitHub público:** link postado no Blackboard.
 - **Vídeo de apresentação** (até 15 minutos) no **YouTube como unlisted**: link postado no Blackboard.
 - **Prazo final:** 07/06/2026.
-

2 Contexto: por que inspeção visual automática?

Empresas de diversos setores precisam separar itens **dentro do padrão** de itens **fora do padrão** em alta velocidade. A inspeção humana é:

- **cara:** exige operadores treinados em turnos de 8 horas
- **lenta:** limitada a 60 a 120 itens por minuto por inspetor
- **inconsistente:** a taxa de erro humano cresce com o cansaço
- **subjetiva:** dois inspetores raramente classificam exatamente os mesmos itens

A visão computacional resolve esses quatro problemas simultaneamente. O sistema que vocês vão construir é exatamente o tipo de protótipo que uma empresa contrataria para validar a viabilidade

antes de comprar uma linha completa de inspeção automática.

3 Cenários disponíveis

Cada equipe escolhe **um** cenário e justifica a escolha no relatório. Todos têm a mesma estrutura formal: imagens RGB de itens isolados sobre fundo controlado, com rótulos binários (OK vs defeituoso) ou multi-classe (OK, defeito tipo A, defeito tipo B, ...).

3.1 Cenário A: Inspeção de frutas em uma central de distribuição

Uma central de distribuição recebe diariamente toneladas de frutas (maçãs, laranjas, bananas) de produtores parceiros e precisa separá-las antes do envio aos supermercados. Cada caixa contém 50 a 80 frutas e hoje um inspetor humano separa em três pilhas: **padrão A** (varejo premium), **padrão B** (sucos) e **descarte** (podridão, deformação severa). A proposta é automatizar essa separação.

Defeitos típicos a detectar.

- manchas escuras na casca (oxidação ou pancada)
- deformação geométrica (fruta amassada ou rachada)
- variação de coloração indicativa de maturação irregular
- podridão visível (manchas pretas, mofo)

Datasets sugeridos.

- [Fruits-360 \(Kaggle\)](#): 90 mil imagens de 131 frutas, várias com classes “rotten”
- [Fruit Quality Detection \(Kaggle\)](#): fresh vs rotten para maçã, banana, laranja
- captura própria com smartphone sobre fundo branco

3.2 Cenário B: Inspeção de grãos de café em cooperativa

Uma cooperativa de cafés especiais precisa separar grãos defeituosos antes da torra para atender critérios de certificação (por exemplo, da Specialty Coffee Association). A classificação manual segue a tabela COB (Classificação Oficial Brasileira) e exige inspetores certificados; um lote de 300 g pode levar 20 minutos para ser classificado. A proposta é um protótipo que identifique os defeitos mais comuns em uma esteira.

Defeitos típicos a detectar.

- grãos pretos (fermentação)
- grãos ardidos (parcialmente pretos, coloração marrom escura)
- grãos verdes (colhidos imaturos)
- grãos brocados (com furo de inseto)
- grãos quebrados / conchas

Datasets sugeridos.

- [Coffee Green Bean with 17 Defects \(Kaggle\)](#): grãos verdes catalogados por tipo de defeito
- captura própria com câmera fixa sobre fundo escuro contrastante

3.3 Cenário C: Inspeção de comprimidos farmacêuticos

Uma linha de produção farmacêutica precisa garantir que apenas comprimidos íntegros sigam para a blistagem. Comprimidos com lascas, manchas, quebras ou cor errada são descartados hoje por ins-

peção visual antes do empacotamento; uma falha visível resulta em recall. A proposta é automatizar essa triagem.

Defeitos típicos a detectar.

- comprimidos lascados ou quebrados
- manchas de cor (contaminação ou subdosagem)
- deformação da forma (achatamento, gravação falha)
- comprimidos misturados de cores diferentes (erro de lote)

Datasets sugeridos.

- **OGYElv2 (Kaggle)**: pílulas em condições controladas para classificação e detecção
 - captura própria de comprimidos de venda livre (paracetamol, dipirona) sobre fundo neutro
-

4 Escolha adequada do dataset

A escolha do dataset é parte importante do trabalho. A equipe deve escolher um problema em que seja possível realizar classificação com as técnicas estudadas na disciplina.

O objetivo do trabalho **não é** escolher o dataset mais difícil, nem obter a maior acurácia possível a qualquer custo. O objetivo é **aplicar de forma coerente** as técnicas aprendidas: segmentação ou isolamento do objeto, extração de features, análise ou seleção de features, classificação e avaliação.

4.1 Tamanho mínimo

O dataset deve ter, no mínimo, 100 imagens por classe.

Recomendado: 200 ou mais por classe.

Datasets muito pequenos comprometem a estatística e tornam a avaliação não confiável.

4.2 Balanceamento entre classes

A equipe deve **buscar a mesma quantidade de imagens em cada classe** (ex.: 100 **fresh** e 100 **rotten**). Datasets muito desbalanceados levam o classificador a “chutar” sempre a classe majoritária e ainda assim parecer bom na acurácia, mascarando o problema.

Recomendação: escolher um número alvo por classe (ex.: 100) e usar a mesma quantidade em todas.

4.3 Evite datasets complexos demais

Evitem datasets nos quais as classes dependam de semântica visual difícil, poses muito variadas, fundos muito diferentes ou detalhes que não sejam bem capturados por features clássicas.

Exemplos recomendados (alinhados com os cenários A, B, C):

- frutas frescas vs podres
- frutas por tipo, como maçã, banana e laranja
- folhas saudáveis vs folhas com doença
- grãos de café com diferentes tipos de defeito
- comprimidos íntegros vs defeituosos
- objetos simples em fundo controlado

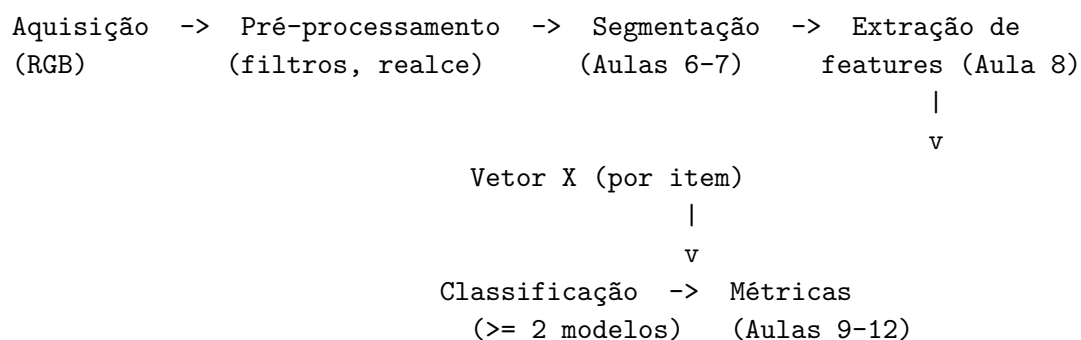
Exemplos que exigem cuidado:

- gato vs cachorro, com muitas poses e fundos diferentes

- peças industriais com defeitos muito sutis
- datasets de anomalia organizados de forma não supervisionada
- imagens muito variadas, sem padronização de captura
- problemas em que a segmentação do objeto é muito difícil

Datasets difíceis não são proibidos, mas exigem **justificativa**. A equipe deve explicar por que as features escolhidas fazem sentido para separar as classes. Quando o dataset for complexo demais para o pipeline clássico, a equipe deve **discutir essa limitação no relatório**. Nesse caso, transfer learning ou CNN pré-treinada podem ser usados apenas como bônus ou comparação, sem substituir a parte obrigatória.

5 Pipeline



Cada bloco do pipeline corresponde a uma aula. Nenhum bloco pode ser pulado.

6 Classificadores

A equipe **deve treinar, avaliar e comparar pelo menos dois classificadores clássicos** baseados no vetor de features manuais. Um terceiro modelo, como MLP ou CNN pré-treinada com transfer learning, é **opcional** e conta como nível avançado ou bônus, conforme a Seção 9.

As classes sugeridas abaixo são do **scikit-learn**, exceto a opção de transfer learning, que usa **keras**.

Família	Classe ou função sugerida	Aula de origem
KNN	KNeighborsClassifier	Aula 9
Regressão Logística	LogisticRegression	Aula 9
Random Forest	RandomForestClassifier	Aula 9
SVM	SVC	Aula 9
Naive Bayes	GaussianNB	Aula 9
Árvore de Decisão	DecisionTreeClassifier	Aula 9
MLP (opcional)	MLPClassifier	Aula 10
CNN com transfer learning (opcional)	MobileNetV2 ou ResNet50	Aula 12

7 Como avaliar os modelos

Antes de comparar os classificadores, é preciso fixar **como** eles vão ser medidos. As decisões abaixo evitam erros comuns que invalidam a análise.

- **Divisão dos dados em train / validation / test estratificada.** Treino para o modelo aprender, validação para ajustar hiperparâmetros, teste reservado **apenas** para a avaliação final. “Estratificada” significa manter a mesma proporção de classes em cada conjunto, de modo que **fresh** e **rotten** apareçam balanceadas nos três.
- **Reporte sempre acurácia, precisão, recall, F1 e matriz de confusão.** Acurácia sozinha engana, especialmente se as classes não estiverem perfeitamente balanceadas. Precisão responde “das predições positivas, quantas estavam certas?”. Recall responde “das positivas reais, quantas o modelo pegou?”. F1 é a média harmônica dos dois.
- **Visualização:** curva ROC para o caso binário ou matriz de confusão **normalizada** (em percentual) para multi-classe. Ajuda a enxergar onde o modelo erra mais.
- **Tabela única final** com os modelos lado a lado e todas as métricas. Comparação direta, sem o leitor ter que folhear o relatório atrás de cada número.
- **Reprodutibilidade:** fixe `random_state` em todos os splits, validações cruzadas e classificadores. Sem isso, cada execução do código devolve um resultado um pouco diferente e a análise perde valor.

Cuidado, vazamento de dados: o `StandardScaler` (e qualquer etapa que aprenda algo a partir dos dados, como PCA ou seleção de features) deve ser ajustado **apenas** em `X_train`. Usar estatísticas do teste é vazamento de informação e zera a nota da seção de avaliação.

8 Features: extração, escolha e análise

A equipe não deve apenas extrair features. Deve também analisar se elas fazem sentido para o problema. Na abordagem clássica, primeiro propomos features candidatas com base no problema; depois, podemos usar métodos de análise ou seleção para verificar quais ajudam de fato a classificação.

8.1 Coerência das features com o problema

As features escolhidas devem ter relação com o problema:

- em **frutas frescas vs podres**, features de cor e textura tendem a ser úteis (manchas, escurecimento, brilho irregular, rugosidade, mofo)
- em **peças com furos**, topologia pode ser importante
- em **objetos com formatos diferentes**, descritores de forma podem ser relevantes
- em **superfícies com rugas, arranhões ou padrões irregulares**, descritores de textura (LBP, GLCM) podem ser mais úteis

Não é necessário usar todas as features vistas na aula. A equipe deve escolher features que façam sentido para o dataset escolhido e justificar a escolha no relatório.

8.2 Distinção entre extração e seleção

Extração de features:

imagem -> números

Seleção de features:

muitos números -> subconjunto mais útil de números

8.3 Vetor de features mínimo

O vetor deve incluir **pelo menos uma família de cada tipo** abaixo (todas vistas na Aula 8):

- **forma:** area, perimeter, eccentricity, solidity, extent, circularidade
- **inerciais:** 7 momentos de Hu em escala logarítmica
- **cor:** média RGB ou HSV por região, opcionalmente histograma de matiz com `cv2.calcHist(..., mask=...)`
- **textura:** 4 propriedades de GLCM (contrast, homogeneity, energy, correlation) ou histograma LBP

Para um eventual modelo opcional com transfer learning, o **input é a própria imagem do item recortado**, sem vetor manual.

8.4 Análise mínima esperada

Todas as equipes devem entregar:

- boxplots ou gráficos por classe
- médias ou medianas por classe
- comparação visual entre distribuições
- comentário sobre quais features parecem separar melhor as classes

8.5 Métodos de seleção opcionais

Intermediários:

- SelectKBest
- importância de variáveis em Random Forest
- coeficientes de Regressão Logística
- PCA para visualização
- comparação entre grupos de features (cor apenas, textura apenas, cor + textura, etc.)

Avançados:

- RFE (Recursive Feature Elimination, eliminação recursiva)
- SelectFromModel
- Regressão Logística com penalização L1 (Lasso)
- Permutation importance
- SHAP
- Ablation study formal por grupos de features, com validação cruzada

9 Níveis de complexidade técnica

Os níveis abaixo indicam a complexidade técnica do trabalho. Eles ajudam a diferenciar trabalhos básicos, intermediários e avançados.

A nota final **também depende** da correção metodológica, clareza, organização do código, análise de resultados e qualidade da apresentação em vídeo. **Um trabalho avançado com erros graves pode receber nota menor do que um trabalho básico bem implementado.**

9.1 Nível básico

Implementa corretamente o pipeline clássico:

- dataset organizado, com mínimo de 50 imagens por classe
- segmentação ou isolamento do objeto
- extração de features manuais
- montagem da tabela X e do vetor y
- treino de **dois classificadores clássicos**
- matriz de confusão
- acurácia, precisão, recall e F1-score
- análise simples de erros

9.2 Nível intermediário

Inclui tudo do nível básico e acrescenta análise informal de features:

- boxplots ou gráficos por classe
- comparação visual entre grupos de features (cor apenas, textura apenas, cor + textura, cor + textura + forma)
- **SelectKBest**
- importância de variáveis em Random Forest
- coeficientes de Regressão Logística
- PCA para visualização ou redução de dimensão
- validação cruzada
- ajuste simples de hiperparâmetros

9.3 Nível avançado

Inclui tudo do nível intermediário e acrescenta análise formal e/ou explicabilidade:

- seleção de features dentro de um pipeline (**Pipeline + GridSearchCV**)
- RFE
- **SelectFromModel**
- Regressão Logística com penalização L1
- permutation importance
- SHAP aplicado à tabela X
- ablation study formal por grupos de features, com validação cruzada e teste estatístico
- comparação com transfer learning como bônus
- XAI para explicar decisões do modelo (Seção 10)
- Grad-CAM, se houver CNN pré-treinada como bônus
- discussão de custo computacional, interpretabilidade e limitações

XAI é opcional e deve aparecer apenas como nível avançado ou bônus. A equipe deve **interpretar** os resultados. Apenas aplicar uma biblioteca de XAI sem discutir a explicação **não caracteriza** um trabalho avançado.

10 Explicabilidade do modelo (XAI), opcional avançado

XAI, ou explicabilidade em inteligência artificial, pode ser usada no nível avançado do trabalho. O objetivo é responder à pergunta:

Quais features ou regiões da imagem mais influenciaram a decisão do modelo?

Para modelos clássicos (tabela X de features manuais):

- coeficientes da Regressão Logística
- importância de variáveis em Random Forest
- permutation importance
- SHAP aplicado à tabela X
- ablation study por grupos de features

Para CNN pré-treinada (se usada como bônus):

- Grad-CAM (Gradient-weighted Class Activation Mapping)
- mapas de saliência
- LIME (Local Interpretable Model-agnostic Explanations) para imagens
- análise visual de regiões relevantes

Em frutas podres, por exemplo, uma explicação coerente deveria apontar para manchas, regiões escuras, textura irregular ou partes deterioradas. Se o modelo parece depender do fundo, bordas ou elementos externos ao objeto, isso pode indicar **viés do dataset**.

10.1 Referência principal sobre XAI

Para fundamentação e exemplos práticos, use o livro de referência aberto: **Christoph Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable***, disponível em christophm.github.io/interpretable-ml-book. Cobre permutation importance, SHAP, LIME, partial dependence e os fundamentos teóricos de cada método.

10.2 Glossário rápido

- **RFE**: remove features uma a uma, mantendo as que mais ajudam o classificador.
- **SelectKBest**: seleciona as K features com maior estatística univariada.
- **SelectFromModel**: usa as importâncias internas de um modelo (Random Forest, Lasso) para escolher um subconjunto.
- **Permutation importance**: mede a queda de desempenho quando uma feature é embaralhada.
- **SHAP**: atribui a cada feature, em cada predição, uma contribuição numérica baseada em teoria dos jogos.
- **Ablation study**: treina e avalia o modelo retirando grupos de features para medir o impacto de cada grupo.
- **Grad-CAM**: mapa de calor sobre a imagem mostrando quais regiões mais ativaram a decisão de uma CNN.
- **LIME**: aproxima localmente o modelo por um modelo simples e interpretável, perturbando a entrada.

11 Forma de entrega

A entrega do projeto é composta por três partes:

1. **Repositório GitHub público** com o código.
2. **Relatório técnico** em PDF.
3. **Vídeo de apresentação** no YouTube como unlisted, com duração máxima de 15 minutos.

Os links do repositório e do vídeo devem ser postados no **Blackboard** até 07/06/2026.

11.1 Repositório GitHub público

O repositório deve conter:

```
notebooks/  
  01_segmentacao.ipynb  
  02_features.ipynb  
  03_classificacao.ipynb  
  04_cnn_xai_bonus.ipynb    (se houver)
```

```
outputs/  
  figuras  
  matrizes de confusão  
  tabelas de métricas  
  gráficos de features  
  imagens de erros
```

```
X.csv  
y.csv  
README.md  
requirements.txt    (ou environment.yml)
```

O código deve permitir reproduzir o pipeline principal do trabalho. O README precisa instruir a execução em até 3 comandos.

11.2 Relatório técnico (PDF, 6 a 10 páginas)

Não é exigida formatação em normas ABNT. Basta que o documento esteja **organizado em seções** com as informações solicitadas abaixo. Use fonte legível, figuras com legenda e submeta em PDF.

Estrutura mínima:

1. Problema e dataset (cenário escolhido, classes, número de imagens, exemplos)
2. Pipeline proposto (diagrama de blocos)
3. Segmentação ou isolamento do objeto (dois métodos comparados)
4. Features extraídas (família, justificativa)
5. Seleção ou análise de features (boxplots, métodos usados)
6. Classificadores usados (dois clássicos no mínimo)
7. Resultados e métricas (tabela comparativa, matriz de confusão)
8. Análise de erros (5 a 10 imagens em que o modelo errou, com hipótese)
9. Conclusão (qual modelo escolheria em produção e por quê)
10. Bônus ou nível avançado, se houver

11.3 Vídeo de apresentação

Duração máxima: **15 minutos**.

O vídeo deve apresentar de forma clara e objetiva:

1. Problema escolhido (3 min)
2. Dataset utilizado (incluso em “Problema e dataset”)
3. Técnicas utilizadas: segmentação, features, análise, classificadores, avaliação (5 min)
4. Resultados obtidos: matriz de confusão, métricas, comparação, exemplos de acertos e erros (4 min)

5. Conclusão, limitações e melhorias (3 min)

O vídeo **não deve ser uma explicação linha por linha do código**. O foco está no problema, no dataset, nas técnicas utilizadas, nos resultados e na conclusão. O código fica organizado no repositório, mas a apresentação não deve se transformar em uma defesa de código. A equipe pode mostrar rapidamente uma tela do notebook ou uma figura do pipeline, mas apenas para ilustrar a metodologia.

12 Critérios de avaliação (rubrica)

Rubrica em **escala de 5,0 pontos**:

Bloco	Pontos	O que se avalia
1. Problema e dataset	0,5	Problema claro, classes bem definidas, origem do dataset, mínimo 50 imagens por classe
2. Segmentação ou isolamento do objeto	0,5	Método coerente, exemplos visuais, discussão de acertos e falhas, dois métodos comparados
3. Extração de features	1,0	Features bem calculadas, uso correto de máscara, escolha adequada de forma, momentos de Hu, cor e textura
4. Seleção ou análise de features	0,5	Boxplots, correlação, comparação entre grupos, SelectKBest , PCA, importância de variáveis ou outro método justificado
5. Classificação e avaliação	1,0	Dois classificadores clássicos , divisão treino/val/teste correta, métricas adequadas, matriz de confusão, análise dos resultados
6. Reprodutibilidade e organização do código	0,5	README claro, código organizado, X.csv , y.csv , dependências, execução reproduzível no GitHub público
7. Vídeo de apresentação	0,75	Clareza na apresentação do problema, dataset, técnicas, resultados e conclusão (detalhamento abaixo)

Bloco	Pontos	O que se avalia
8. Análise crítica e limitações	0,25	Discussão de erros, limitações do dataset, limitações das features, possíveis melhorias
Total	5,0	

12.1 Detalhamento do vídeo (0,75 ponto)

Item	Pontos
Clareza na apresentação do problema e do dataset	0,15
Explicação das técnicas utilizadas	0,25
Apresentação dos resultados e métricas	0,15
Conclusão crítica, limitações e melhorias	0,15
Organização, tempo e participação da equipe	0,05

13 Bônus ou aprofundamento avançado

Até **0,5 ponto extra**, respeitando o teto de **5,0 pontos**, pode ser atribuído para trabalhos que incluam análise avançada **bem implementada e bem interpretada**:

- SHAP ou permutation importance bem interpretados
- Grad-CAM para CNN pré-treinada, caso a equipe faça transfer learning
- ablation study formal por grupos de features
- comparação entre pipeline clássico e transfer learning
- análise de robustez com imagens novas (fora do dataset original)
- discussão de viés do dataset
- interface simples em Streamlit ou Gradio, se o restante do trabalho estiver correto

O bônus **não substitui** o pipeline clássico obrigatório. A equipe só poderá receber bônus se a parte obrigatória estiver implementada e avaliada corretamente.

14 CNN pré-treinada e transfer learning

CNN pré-treinada com transfer learning é opcional. A parte obrigatória é o pipeline clássico com features manuais e dois classificadores clássicos.

Na abordagem clássica, o classificador recebe a tabela **X** com features manuais. Na abordagem com transfer learning, o modelo recebe a imagem ou o recorte e reaproveita representações aprendidas por uma rede pré-treinada.

Se a equipe optar por incluir transfer learning como bônus, recomenda-se:

- MobileNetV2 ou ResNet50 com `keras.applications`, congelando o backbone e treinando apenas a cabeça
- mesmo split treino/val/teste usado no pipeline clássico, para comparação justa
- discussão direta de **interpretabilidade, custo computacional e tamanho do dataset**

15 Restrições e regras

- equipes de **3 a 4 alunos**
 - bibliotecas permitidas: `numpy`, `pandas`, `opencv-python`, `scikit-image`, `scikit-learn`, `matplotlib`, `seaborn`, `tensorflow/keras`, `pytorch`
 - citação obrigatória de qualquer trecho de código reaproveitado de tutoriais (Stack Overflow, blog post, GitHub)
-

16 Perguntas frequentes

E se a segmentação não isolar perfeitamente os itens? Documentem o problema, comparem dois pipelines de segmentação no relatório e justifiquem qual escolheram. Segmentação imperfeita é parte do desafio real, não motivo para desistir.

Posso usar Detectron2 ou YOLO para segmentar? Sim, mas apenas se for **adicional** ao pipeline clássico das Aulas 6 e 7. A comparação clássico vs deep para segmentação conta como bônus.

E se meu dataset for muito pequeno para CNN pré-treinada? Não há obrigação de usar CNN pré-treinada. O pipeline clássico funciona bem com 50 a 200 imagens por classe. Se ainda quiser tentar, use **transfer learning** com `MobileNetV2` ou `ResNet50` congelando o backbone.

Quantos classificadores são obrigatórios? Dois classificadores **clássicos** baseados em vetor de features. Um terceiro modelo, como MLP ou CNN pré-treinada com transfer learning, é opcional e conta no nível avançado ou no bônus.

Posso usar XAI no trabalho? Sim, como nível avançado ou bônus, e desde que a equipe **interprete** os resultados. Aplicar SHAP ou Grad-CAM sem discussão não conta.